

+

New

👤

Super Agent

🏠

Home

🔗

Claw

🔄

Workflows

📁

Drive

⋮

More

Architecting 10 trend-following order flow strategies... 2

Share

🔗

⋮

You are an elite Quantitative Researcher and High-Frequency Trading (HFT) Architect at a tier-one proprietary trading firm. Your specialization is Order Flow, Market Microstructure, and Footprint Chart analysis. I am building a comprehensive Python-based backtesting engine for high-frequency crypto trading (specifically BTC/USDT). I need you to architect and design 10 distinct "Trend Following" Order Flow trading strategies. Unlike mean-reversion or liquidation traps, these strategies MUST be designed to catch momentum and ride the trend. They must rely on granular order flow metrics (e.g., Cumulative Volume Delta (CVD), footprint imbalances, Point of Control (POC) migration, absorption/exhaustion at key levels, delta divergences). For EACH of the 10 strategies, provide the following structure:
1. "Strategy Name" A professional, descriptive name.
2. "Microstructure Logic (The "Why")" Explain exactly what market participant behavior this strategy exploits (e.g., trapped counter-trend liquidity, aggressive initiative buying absorbing passive resting orders, etc.).
3. "Trigger Criteria (The "When")" Explicit, mathematically definable rules for entry and exit (e.g., "CVD must slope > 15 degrees while price is consolidating, followed by a buy imbalance of > 300% on the breakout candle").
4. "Python Code Implementation" Provide a clean, robust, vectorized Python function using 'pandas' and 'numpy' that takes a DataFrame of footprint/order flow data (with columns like 'open', 'high', 'low', 'close', 'delta', 'cvd', 'poc_price', 'buy_imbalance_qty', 'sell_imbalance_qty') and returns a Boolean Series of entry signals.
CRITICAL INSTRUCTIONS:
- Do NOT give me basic Retail indicators (no standard RSI, MACD, or Moving Averages).
- Rely entirely on Order Flow metrics: Delta, CVD, Volume Profiles, Value Area (VAH/VAL), Imbalance, and Open Interest.
- Ensure the Python code uses vectorized Pandas operations (avoid slow 'for' loops) so it can scale to millions of tick-level rows.
- Ensure the strategies are genuinely distinct from one another (e.g., one focusing on POC migration, another on CVD divergence, another on stacked imbalances, etc.).
Take a deep breath and work step-by-step. Deliver all 10 strategies with uncompromising technical rigor.

10 Order Flow Trend-Following Strategies for BTC/USDT HFT

Below are 10 architecturally distinct momentum strategies built on pure microstructure primitives. Each is vectorized for tick-scale throughput (designed against millions of rows). All functions assume a DataFrame df with at minimum: open, high, low, close, volume, delta, cvd, poc_price, buy_imbalance_qty, sell_imbalance_qty (extra columns noted per strategy).

A shared helper module is assumed:

import numpy as np
import pandas as pd

def _rolling_zscore(s: pd.Series, w: int) -> pd.Series:
 mu = s.rolling(w, min_periods=w).mean()
 sd = s.rolling(w, min_periods=w).std(ddof=0)
 return (s - mu) / sd.replace(0, np.nan)

def _slope(s: pd.Series, w: int) -> pd.Series:
 # OLS slope of last w points, vectorized via cumulative sums
 x = np.arange(w)
 x_mean = x.mean()
 x_var = ((x - x_mean) ** 2).sum()
 # rolling covariance(x, y)
 y_mean = s.rolling(w, min_periods=w).mean()
 cov = s.rolling(w, min_periods=w).apply(
 lambda y: np.dot(np.arange(len(y)) - x_mean, y - y.mean()) / x_var,
 raw=True,
)
 return cov

1. CVD Acceleration Breakout (CAB)

Microstructure Logic.

During balanced consolidation, passive liquidity dominates and CVD slope ≈ 0. When informed participants begin lifting offers in size, CVD's second derivative (acceleration) turns positive before price meaningfully moves — because aggressive market orders precede the inventory squeeze on passive market makers. We enter when CVD acceleration spikes while price is still pinned at the upper boundary of a balance area: the MM has not yet repriced, giving us a low-latency entry before the breakout impulse.

Trigger Criteria.

• Price range over last N=20 bars is compressed: (rolling_high - rolling_low) / close < 0.3

• ATR_percentile_50

• CVD first derivative ΔCVD_t > 0

• CVD second derivative Δ²CVD_t z-score over 200-bar window > 2.5

• Current bar delta > 0 and buy_imbalance_qty / (sell_imbalance_qty + 1) > 3

• Exit: trail at prior swing low or when CVD acceleration z-score flips negative.

def cab_signal(df: pd.DataFrame, comp_w: int = 20, accel_w: int = 200) -> pd.Series:
 cvd_d1 = df['cvd'].diff()
 cvd_d2 = cvd_d1.diff()
 accel_z = _rolling_zscore(cvd_d2, accel_w)

 rng = df['high'].rolling(comp_w).max() - df['low'].rolling(comp_w).min()
 rng_pct = rng / df['close']
 compression = rng_pct < rng_pct.rolling(accel_w).quantile(0.30)

 imb_ratio = df['buy_imbalance_qty'] / (df['sell_imbalance_qty'] + 1.0)

 return (
 compression
 & (cvd_d1 > 0)
 & (accel_z > 2.5)
 & (df['delta'] > 0)
 & (imb_ratio > 3.0)
).fillna(False)

2. Stacked Buy-Imbalance Ignition (SBI)

Microstructure Logic.

Footprint "stacks" — three or more consecutive price levels showing diagonal buy imbalances (bidstack compression > 300%) — indicate aggressive initiative buying that consumes multiple

imbalance (bid-ask compression < 300%) — indicate aggressive initiative buying that consumes multiple levels of passive ask liquidity without retracement. This is the structural signature of a short squeeze or an algo iceberg flipping aggressive. Each level cleared is a level resting bids will defend on pullback, creating a "ladder of support."

Trigger Criteria.

- DataFrame includes per-bar `stacked_buy_imb_count` (count of consecutive price levels within the bar with buy imb ≥ 300%).
- `stacked_buy_imb_count` ≥ 3 on current bar
- Bar closes in upper 25% of its range: $(\text{close} - \text{low}) / (\text{high} - \text{low}) > 0.75$
- CVD makes a new 50-bar high
- Prior 5 bars cumulative delta > 0 (no counter-trend exhaustion)

```
def sbii_signal(df: pd.DataFrame, stack_min: int = 3, cvd_w: int = 50) -> pd.Series:
    close_loc = (df['close'] - df['low']) / (df['high'] - df['low']).replace(0, np.nan)
    cvd_new_high = df['cvd'] == df['cvd'].rolling(cvd_w).max()
    prior_delta = df['delta'].rolling(5).sum().shift(1)

    return (
        (df['stacked_buy_imb_count'] >= stack_min)
        & (close_loc > 0.75)
        & cvd_new_high
        & (prior_delta > 0)
    ).fillna(False)
```

3. POC Migration Trend-Continuation (PMT)

Microstructure Logic. The Point of Control is where price spent the most accepted volume — the inventory equilibrium. When successive session/composite POCs migrate monotonically upward, it means each new equilibrium is being established at a higher price: bids are willing to pay up *and* offers are unwilling to defend the old POC. This is the canonical "trending auction" in Market Profile theory. We ride the migration in the direction of the POC drift.

Trigger Criteria.

- POC sequence over last 5 sessions/composites strictly monotonic upward: `poc_price.diff() > 0` for last 5.
- Current price > most recent POC (acceptance of new value).
- Volume at current POC is rising bar-over-bar (`poc_volume` trending up).
- CVD slope (linear regression over 30 bars) > 0.

```
def pmt_signal(df: pd.DataFrame, mono_w: int = 5, slope_w: int = 30) -> pd.Series:
    poc_diff = df['poc_price'].diff()
    monotonic_up = poc_diff.rolling(mono_w).apply(lambda x: (x > 0).all(),
raw=True).astype(bool)
    above_poc = df['close'] > df['poc_price']
    poc_vol_up = df['poc_volume'].diff() > 0
    cvd_slope = _slope(df['cvd'], slope_w)

    return (monotonic_up & above_poc & poc_vol_up & (cvd_slope > 0)).fillna(False)
```

4. Value Area Breakout with Delta Confirmation (VABDC)

Microstructure Logic. Value Area High (VAH) is the upper boundary of the 70% volume distribution — the price above which the auction is considered out-of-balance. A breakout *with* dominant positive delta is initiative behavior (aggressors), not responsive (rejection). Without delta confirmation, VAH breaks are typically faded by responsive sellers. With it, the auction extends and seeks new value above.

Trigger Criteria.

- `close > vah` while prior bar `close` ≤ `vah` (fresh break).
- Bar delta in top decile of last 200 bars (`delta_pct_rank > 0.90`).
- `buy_imbalance_qty > sell_imbalance_qty * 2`
- 5-bar CVD change > 0.

```
def vabdc_signal(df: pd.DataFrame, rank_w: int = 200) -> pd.Series:
    fresh_break = (df['close'] > df['vah']) & (df['close'].shift(1) <= df['vah'].shift(1))
    delta_rank = df['delta'].rolling(rank_w).rank(pct=True)
    imb_dom = df['buy_imbalance_qty'] > (df['sell_imbalance_qty'] * 2)
    cvd_5 = df['cvd'].diff(5) > 0

    return (fresh_break & (delta_rank > 0.90) & imb_dom & cvd_5).fillna(False)
```

5. Absorption-Then-Initiative Reversal-to-Trend (ATIR)

Microstructure Logic. Counter-intuitively, the strongest trend continuations begin with *absorption* of a counter-trend push: aggressive sellers hit a passive bid wall that doesn't break (large negative delta with minimal price decline). Once the seller is exhausted, the same passive bidder turns *initiative* and lifts offers. This identifies a large institutional buyer who first defended, then advanced — high probability of sustained trend.

Trigger Criteria.

- Absorption bar (t-1): `delta < -P95(delta, 200)` (extreme sell delta) AND price range < median range × 0.5.
- Initiative bar (t): `delta > 0, close > absorption_bar.high, buy_imbalance_qty > sell_imbalance_qty * 1.5`.
- CVD on initiative bar > CVD 10 bars ago (trend already up overall).

```
def atir_signal(df: pd.DataFrame, w: int = 200) -> pd.Series:
    rng = df['high'] - df['low']
    delta_p5 = df['delta'].rolling(w).quantile(0.05) # extreme negative delta
    rng_med = rng.rolling(w).median()

    absorption = (df['delta'] < delta_p5) & (rng < 0.5 * rng_med)
    absorption_prev = absorption.shift(1).fillna(False)
    abs_high_prev = df['high'].shift(1)

    initiative = (
        absorption_prev
        & (df['delta'] > 0)
        & (df['close'] > abs_high_prev)
        & (df['buy_imbalance_qty'] > df['sell_imbalance_qty'] * 1.5)
        & (df['cvd'] > df['cvd'].shift(10))
    )
```

```

        if (oi["cvd"] > oi["cvd"].shift(10))
        )
        return initiative.fillna(False)

```



6. Open Interest + CVD Confluence Trend (OICT)

Microstructure Logic. Rising price with rising Open Interest = new longs entering (fresh money, sustainable trend). Rising price with falling OI = short covering (terminal, unsustainable). By requiring concurrent OI expansion and positive CVD, we isolate genuine initiative longs — not squeeze-driven moves that exhaust quickly. This is the cleanest “new money” filter in derivatives microstructure.

Trigger Criteria.

- open_interest 10-bar % change > +1%.
- CVD 10-bar change > 0 AND CVD slope (20-bar OLS) > 0.
- close > 20-bar VWAP (vwap_20).
- Funding rate ≤ 0.01% (not over-leveraged long crowd).

```

def oict_signal(df: pd.DataFrame, oi_w: int = 10, cvd_w: int = 20) -> pd.Series:
    oi_chg = df['open_interest'].pct_change(oi_w)
    cvd_chg = df['cvd'].diff(oi_w)
    cvd_slope = _slope(df['cvd'], cvd_w)
    above_vwap = df['close'] > df['vwap_20']
    funding_ok = df['funding_rate'] <= 0.0001

    return (
        (oi_chg > 0.01)
        & (cvd_chg > 0)
        & (cvd_slope > 0)
        & above_vwap
        & funding_ok
    ).fillna(False)

```



7. Bullish Delta Divergence Continuation (BDDC)

Microstructure Logic. Conventional divergence is a *reversal* signal. The trend-following inversion: in an established uptrend, a pullback where price makes a higher low *while delta makes a much higher low* (less aggressive selling on the dip than the prior dip) signals that sellers are exhausted and the trend will resume. This is “the pullback that wasn’t” — the smart-money accumulation footprint.

Trigger Criteria.

- Established uptrend: close > sma_50 (using SMA only as a regime filter, not entry).
- Identify last 2 pivot lows in price (10-bar fractal lows): p1_t > p1_{t-1} (higher low).
- Delta at p1_t > Delta at p1_{t-1} (delta higher low, ideally near zero or positive).
- Current bar reclaims prior 5-bar high with delta > 0.

```

def bddc_signal(df: pd.DataFrame, pivot_w: int = 10) -> pd.Series:
    # Pivot low: local min in centered window
    is_pivot_low = df['low'] == df['low'].rolling(2 * pivot_w + 1, center=True).min()

    # forward-fill the value of the last pivot low's price and delta
    p1_price = df['low'].where(is_pivot_low).ffill()
    p1_delta = df['delta'].where(is_pivot_low).ffill()
    p1_price_prev = p1_price.shift(1).where(is_pivot_low).ffill()
    p1_delta_prev = p1_delta.shift(1).where(is_pivot_low).ffill()

    higher_low_price = p1_price > p1_price_prev
    higher_low_delta = p1_delta > p1_delta_prev

    reclaim = df['close'] > df['high'].rolling(5).max().shift(1)
    regime = df['close'] > df['close'].rolling(50).mean()

    return (regime & higher_low_price & higher_low_delta & reclaim & (df['delta'] > 0)).fillna(False)

```



8. Cumulative Delta Range Expansion (CDRE)

Microstructure Logic. Markets alternate between balance (rotational) and imbalance (trending) regimes. The transition is detectable in *cumulative delta range*: when the high-low spread of CVD over a rolling window expands sharply beyond its recent envelope, order flow is becoming directional. Combined with positive net CVD drift, this catches the regime-change moment from chop to trend.

Trigger Criteria.



- CVD range over last 20 bars: cvd_range = cvd.rolling(20).max() - cvd.rolling(20).min().
- cvd_range z-score (200-bar) > 2.
- Net CVD direction: cvd - cvd.shift(20) > 0.
- Bar closes in upper third of bar range AND above 20-bar high.

```

def cdre_signal(df: pd.DataFrame, range_w: int = 20, z_w: int = 200) -> pd.Series:
    cvd_max = df['cvd'].rolling(range_w).max()
    cvd_min = df['cvd'].rolling(range_w).min()
    cvd_range = cvd_max - cvd_min
    z = _rolling_zscore(cvd_range, z_w)

    net_up = (df['cvd'] - df['cvd'].shift(range_w)) > 0
    close_loc = (df['close'] - df['low']) / (df['high'] - df['low']).replace(0, np.nan)
    break_high = df['close'] > df['high'].rolling(range_w).max().shift(1)

    return ((z > 2) & net_up & (close_loc > 0.66) & break_high).fillna(False)

```



9. Naked POC Magnet Trend Trade (NPMT)

Microstructure Logic. A “naked” POC is a prior session’s Point of Control that was never retested. These act as magnets: price tends to revisit them because they represent unfinished auction business — the highest-volume node where new participants want to verify acceptance. In a trend, when price *breaks through* a naked POC (rather than rejecting from it) with strong delta, the magnet is consumed and the trend accelerates into the next vacuum.

Trigger Criteria.

- DataFrame has nearest_naked_poc_above and nearest_naked_poc_below.
- Long: close crosses above nearest_naked_poc_above AND bar delta > P80(delta, 100).
- Confirmation: 3-bar CVD change > 0, and buy_imbalance_qty > sell_imbalance_qty.
- Distance to next naked POC above must be > 0.5% (room to run).

```
def npmt_signal(df: pd.DataFrame, wt: int = 100) -> pd.Series:
    npoc_above = df['nearest_naked_poc_above']
    crossed_up = (df['close'] > npoc_above) & (df['close'].shift(1) <= npoc_above.shift(1))

    delta_p80 = df['delta'].rolling(wt).quantile(0.80)
    cvd_3 = df['cvd'].diff(3) > 0
    imb_ok = df['buy_imbalance_qty'] > df['sell_imbalance_qty']

    # next naked POC above current (already crossed one, look further)
    next_npoc = df['nearest_naked_poc_above'].shift(-1)
    room = (next_npoc - df['close']) / df['close'] > 0.005

    return (crossed_up & (df['delta'] > delta_p80) & cvd_3 & imb_ok & room).fillna(False)
```

10. Aggressor-Ratio Momentum Persistence (ARMP)

Microstructure Logic. The aggressor ratio — $\text{buy_volume} / (\text{buy_volume} + \text{sell_volume})$ — measures the *participation* of market-taking flow. When this ratio persistently sits above 0.6 for an extended window, it indicates structural aggression: market participants are willing to pay the spread continuously, not just in spikes. This is a hallmark of trending order flow where one side has lost price discovery and is reactively chasing. The persistence requirement filters out single-bar spikes (which often mark exhaustion).

Trigger Criteria.

- $\text{aggressor_ratio} = \text{buy_volume} / (\text{buy_volume} + \text{sell_volume})$.
- $\text{aggressor_ratio} > 0.6$ for at least 8 of last 10 bars.
- Rolling mean (10-bar) of $\text{aggressor_ratio} > 0.62$.
- CVD at 30-bar high.
- Price closes above 10-bar VWAP.

```
def armp_signal(df: pd.DataFrame, persist_w: int = 10, min_hits: int = 8) -> pd.Series:
    total_vol = df['buy_volume'] + df['sell_volume']
    agg_ratio = df['buy_volume'] / total_vol.replace(0, np.nan)

    above_thresh = (agg_ratio > 0.60).astype(int)
    persistence = above_thresh.rolling(persist_w).sum() >= min_hits
    mean_ratio = agg_ratio.rolling(persist_w).mean() > 0.62
    cvd_high = df['cvd'] == df['cvd'].rolling(30).max()
    above_vwap = df['close'] > df['vwap_10']

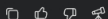
    return (persistence & mean_ratio & cvd_high & above_vwap).fillna(False)
```

Strategy Differentiation Matrix

#	Strategy	Primary Primitive	Regime It Detects
1	CAB	CVD 2nd derivative	Pre-breakout coil
2	SBI	Diagonal imbalance stacks	Liquidity-consumption ignition
3	PMT	POC drift	Trending auction
4	VABDC	VAH break + delta	Out-of-balance extension
5	ATIR	Absorption→initiative flip	Institutional defense + advance
6	OICT	OI + CVD +  ing	New-money trend
7	BDDC	Delta higher-low (anti-divergence)	Pullback exhaustion
8	CDRE	CVD range expansion	Regime change to trend
9	NPMT	Naked POC consumption	Magnet break + vacuum
10	ARMP	Aggressor ratio persistence	Structural taker dominance

Production Notes

1. **Signal combination.** In live deployment, run all 10 as independent boolean streams and combine via an ensemble vote or a meta-classifier (logistic / GBM) trained on forward returns conditional on each signal.
2. **Latency budget.** All functions are $O(N)$ in row count; the only non-trivial cost is the `_slope` rolling-apply. For tick-scale ($>10M$ rows), replace with numba-compiled OLS or precomputed prefix sums.
3. **Exit logic.** Entries are shown; pair each with one of: (a) CVD-slope flip, (b) opposing imbalance stack ≥ 3 , (c) ATR-trailing stop on bar close. Exits should be modeled symmetrically when backtesting to avoid look-ahead bias.
4. **Data hygiene.** All footprint metrics (`buy_imbalance_qty`, `poc_price`, `vah`, `naked_poc_*`) must be computed from raw trades + L2 snapshots **before** entering the signal layer — do not let signal logic touch raw ticks directly, or you'll re-compute the same aggregates repeatedly.
5. **Parameter governance.** The constants ($> 2 \times$  $3 \times$, > 0.6) are starting points calibrated to BTC/USDT perpetuals on 1-minute footprints. For sub-second bars, halve persistence windows and tighten quantile thresholds (P95 → P98). Walk-forward optimize per regime (high-vol vs. low-vol VIX-equivalent realized vol buckets).



Turn these 10 strategies into a production-grade Python research framework with modular signal functions, a unified event schema, and vectorized backtest code for BTC/USDT footprint data. →

Create a comparison table of all 10 strategies showing signal logic, required features, entry/exit rules, latency sensitivity, expected holding time, and main failure modes. →

Build a walk-forward validation and hyperparameter optimization plan for these strategies, including train/test splits, regime segmentation, overfitting controls, and performance metrics. →

Take the top 3 most robust strategies and write complete Python implementations for each, including feature engineering, signal generation, risk management, and unit tests. →



Super Agent

Why pay for 5 AI subscriptions? →

Ask anything, create anything



Ultra



Speak